

Received July 9, 2021, accepted July 18, 2021, date of publication July 26, 2021, date of current version August 3, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3100316

A Resource Allocation Model Based on Trust Evaluation in Multi-Cloud Environments

A. B. M. BODRUL ALAM¹, (Member, IEEE),
ZUBAIR MD. FADLULLAH^{1,2}, (Senior Member, IEEE),
AND SALIMUR CHOUDHURY², (Senior Member, IEEE)

¹Thunder Bay Regional Health Research Institute (TBRHRI), Thunder Bay, ON P7B 7A5, Canada

²Department of Computer Science, Lakehead University, Thunder Bay, ON P7B 5E1, Canada

Corresponding author: A. B. M. Bodrul Alam (abalam@lakeheadu.ca)

ABSTRACT Allocating and managing resources while considering the quality of service is considered a fundamental and complex research problem in a cloud environment. An optimal resource allocation optimizes several parameters such as optimizing cost and resource utilization or maximizing any quality parameters. However, to ensure better customer service, Cloud Service Providers (CSPs) should consider most of the quality attributes while allocating resources to the cloud infrastructures. Existing research does not evaluate trust as a quantitative attribute, thus a trade-off between trust and performance in resource allocation is also absent in the research area. We propose a model to consider both trust and delay in this paper. The trust of a CSP is quantitatively estimated through some attributes and metrics. Availability, reliability, data integrity, and efficiency are considered to estimate the trust. The objective is to maximize the trust of the allocation while minimizing the communication delay. The proposed joint optimization model combines the previous credentials of the CSPs and the present resource constraints. To solve the problem heuristically, a genetic algorithm is applied. The model uses a number of parameters that provide the flexibility to adapt several service requirements. The effectiveness and applicability of the proposed approach are demonstrated through experiments. The results ensure that the effectiveness in the estimation of trust evaluation for different CSPs with the proposed attributes. Moreover, integrating trust in the resource allocation model allocates appropriate resources while enhancing the trust and reducing the communication delay in the overall allocation.

INDEX TERMS Cloud computing, trust evaluation, resource allocation, availability, reliability, optimization, genetic algorithm.

I. INTRODUCTION

Cloud is a popular computing paradigm and addressed intensively by both academia and industry. Cloud has become popular due to its unique characteristics. Some of the cloud characteristics also negatively impact the quality of service. The unique five characteristics of cloud which improve performance can also increase risks [1]. For instance, there are several applications for which missing any provisioning of services is considered as a failure [2]. Broad network access introduces new challenges. The networking service reliability, availability, and latency directly impact the Quality of Service (QoS) in a cloud environment. Another characteristic is related to rapid elasticity. If the service is not capable of

adapting to the current load then elasticity failure may occur that will impact the reliability of the service. The last feature concerns the measured service. The measured service is the computation of resource usage. If any loss or unavailability of resources occurs, it can lead to unreliable or inaccurate estimation in data-related measurements.

The fundamental characteristic of the cloud is resource sharing by multiple users. Resources in the cloud can be dynamically re-allocated based on users' demands and preferences. Allocating resources appropriately is considered one of the primary concerns in cloud computing. The word appropriate can be any of the performance metrics: cost-effectiveness, accuracy, and reliability. Allocating resource appropriately refers to such an allocation that integrates the resource utilization of the service providers while meeting customer's requirements. Allocation of an

The associate editor coordinating the review of this manuscript and approving it for publication was Ali Kashif Bashir.

optimal resource must avoid resource contention, fragmentation, over-provisioning, and under-provisioning. Resource contention occurs when several applications request the same resources at a specific time. Due to the unavailability of resources, scarcity of resources may happen. Fragmentation of resources may take place for resource isolation. Over-provisioning happens if providers allocate more resources than the demand and under-provisioning may happen if providers allocate fewer resources than its demand. We try to avoid all these cases in our resource allocation framework.

The main obstacles of this paradigm are reliability and security. To address reliability and security both, we propose a model using four quality attributes: availability, reliability, data integrity, and efficiency to estimate the trust value of a CSP. At first, the experiments on the evaluation for the trust model are conducted. After that, the results are integrated into the optimization of the resource allocation model. Our contributions in this paper are threefold:

- Most of the research work in the literature used trust as a quality attribute. In this work, we evaluate trust as a quantitative attribute and estimate the values of trust of different service providers based on our proposed four quality attributes. The values of the quality attributes are estimated based on a set of metrics through a simulation environment. We combine the four quality attributes as a weighted sum to obtain the trust of a provider.
- We integrate the estimated trust value into a resource allocation model in a multi-cloud environment. The resource allocation model also considers the traffic and capacity of the CSPs to evaluate the communication delay. We design the model as a joint optimization problem and aim to maximize trust and minimize communication delay to improve the QoS.
- We solve the resource allocation problem optimally and heuristically. Intlinprog in MATLAB is used as an optimizer to solve the proposed resource allocation model. The optimizer requires several hours to obtain an optimized solution in some scenarios. To avoid such situations, we use a meta-heuristic, a Genetic Algorithm for generating computationally efficient approximated solutions. We compare the results of our heuristics with the optimal solution. The solution of the Genetic algorithm achieves 90% similarity to the exact solution. The execution time of the Genetic Algorithm increases linearly with an increase in the number of the CSPs and servers. These trends ensure the validity of the proposed trust model in a practical cloud environment.

The trust evaluation model considers most of the main elements of a cloud that impact the service performance. Results demonstrated that evaluation of trust and integrating that into the resource allocation model are effectively conducted. The simulation of the model confirms the trust maximization and delay minimization in the allocation. Thereby improves the quality of services. The model introduces flexibility into the resource allocation process as adjusting some parameters can

provide customized users' requirements. As trust and delay are conflicting objectives of the model, the trade-off should be estimated with proper caution.

The organization of the rest of the paper is following. The existing research work regarding the evaluation of quality attributes of cloud, optimization of resource utilization, and application of the meta-heuristic algorithm in cloud resource allocation are discussed in Section II. The proposed trust evaluation model is described in Section III. The resource allocation and problem formulation of our optimization model are presented in Section IV. This section also presents the proposed algorithm to solve the problem. The experimental setup and results analysis are discussed in Section V. Finally, Section VI concludes the paper by mentioning some future research directions.

II. RELATED WORK

The demand for cloud resources changes rapidly and impacts the availability of cloud resources over time. The occurrence of failures is unpredictable and directly impacts the reliability of the cloud. Optimal allocation of resources in a cloud is crucial and can be solved using heuristics. The general process for allocating resources requires three parties: customer, broker, and service provider. At first, customers specify the amounts of resources along with the constraints related to service performance. Then, a broker is responsible for finding a proper CSP or multiple CSPs that can satisfy customers' requirements and transfers the requests to the chosen CSPs. After that, CSPs allocate resources to their data centers based on availability. In such allocation, resource utilization, and customers' requirements both constraints are fulfilled. Managing and optimizing the allocation of resources appropriately in a cloud environment is crucial. There have been many recent efforts to optimize resource allocation and we acknowledge a few of them below.

Analysis of hardware reliability of cloud and the causes of the failures impacting the reliability are discussed in [3]. The authors investigate different parameters such as server age, configuration, location, load executing on the infrastructure and conclude that none of the parameters directly impact the failures significantly. Selecting an optimal channel for improving reliability and minimizing energy budget is the objective of this work [4]. A machine learning approach with reliability improvement and energy awareness is applied in the proposed method. The unique dimensions of resource management in IoT such as managing memory, energy, and process are discussed with pros and cons [5]. A modified Max-Min algorithm is applied for allocating resources in a cloud environment [6]. The modification improves the average waiting time and completion time. The method maximizes the overall resource utilization. A model to evaluate cloud reliability is presented in [7]. The authors use a state transition graph with three states for the evaluation. A Semi-Markov model is used for estimating the reliability using the state transition probability. Some of the software failures are considered in the evaluation. For the physical

resource breakdown, the authors use the mean-time to recovery and failure for availability and downtime. Failures of all main elements of a cloud are not considered in the estimation of cloud reliability. In the model, the authors calculate the reliability of each physical resource there is no focus on performing a reliable allocation of resources for the requested services. A reliability evaluation model for cloud environment is presented in [8]. The model proposes some attributes and metrics for reliability evaluation. Four quality attributes are chosen for calculating the reliability of a cloud. For each of the attributes, the authors proposed some metrics for the estimation and finally combine them using a weighted sum technique. Based on response time, reliability and performance in a cloud are presented in [9]. The response time was calculated based on three parameters: waiting time, service time, and execution time. Queuing theory is applied in the estimation of waiting time and service time. Response time was estimated by combining the above attributes. Faragardi *et al.* [10] proposed an evaluation model for cloud reliability. To evaluate system reliability, they considered the reliability of servers and links.

The primary concern of most of the research is optimizing resource utilization. Resource allocation techniques are discussed with an objective to control various parameters: costs, execution time, and scheduling policy. For instance, Obtaining the minimum servers is the objective of this scheme [11]. The authors propose a heuristic for an allocation that minimizes the requirements of the servers through response time distribution. The effectiveness of the mechanism is evaluated with several performance measurements. The approach does not consider any other parameters that also directly impact service performance. Scheduling task in an efficient manner while minimizing the power consumption is the aim of this work [12]. A prediction mechanism is used for allocating resources and reducing power consumption. A resource allocation problem in a cloud radio access network is shown in the model for choosing the optimal technique to combine all the available spectral bands [13]. A modified version of the cloud radio access network is proposed for handling spectral resources efficiently. Another model for minimizing the server quantity of cloud is presented in [14]. A prediction algorithm that can model the load in advance is applied to predict future resource usages. The authors presented an evaluation to prove the validity of the proposed model. The evaluation shows that it can avoid the occurring of overload and achieve green computing. A resource allocation scheme for monitoring and predicting future requirements is proposed in [15]. The scheme adjusts VMs based on actual resource needs. There are two primary challenges to this task. The first one is when to reallocate resources and the second one is how much resource needs adjustment. However, the performance shows the applicability of the method but adjusting and selecting the threshold is critical and it significantly impacts resource optimization. For a hybrid Cloud, Grewal and Pateria [16] propose a resource provisioning technique. A private cloud is capable of scaling up using

public cloud resources with the help of a resource manager. In allocating resources, most of the approaches are concerned only with resource utilization. To improve the performance, integration of several other performance metrics like reliability, efficiency should also be required into the allocation model.

For an appropriate VM allocation in a cloud, a traffic-aware algorithm is presented in [17]. A swarm optimization algorithm is applied for solving the VM allocation problem. The simulation results show that the algorithm can significantly improve energy consumption and control the loss of communication links. The authors propose a model to maximize resource utilization and minimize energy consumption [18]. A particle swarm optimization method is used to solve the problem. A cloud resource allocation model to improve security is proposed by Halabi *et al.* [19]. The authors designed the allocation problem as an optimization problem and propose a heuristic for the solution. For forecasting the utilization of resources in a data center, a Genetic Algorithm is proposed in [20]. The GA forecasts the requirements of the resources depending on the previous data. A resource allocation approach depending on resource prediction is also proposed in the work. The validity and applicability of the method are presented through simulation results. The results show that GA is superior for an accurate prediction. Moreover, it improves the utilization of CPU and memory and the consumption of energy. Finally, a fault-aware resource allocation for cloud is proposed by Gupta and Ghrera [21]. The authors aim for the maximization of the utilization of resources and consumption of power for the servers.

Most of the research in this area proposes approaches to evaluate some quality attributes and optimize those attributes. The absence of standard quantitative attributes and metrics and the incompleteness in considering the major elements of cloud infrastructure are ensured through the existing cloud literature. No research work in the literature considers most of the major cloud elements in the evaluation to improve QoS. The objective of our approach is to maximize trust and minimize the communication delay to improve service performance. Trust will be calculated based on the availability, reliability, data integrity, and efficiency of a CSP while allocating resources efficiently based on users' requirements. Consideration of trust and communication delay together in the optimization problem makes our model demanding and appropriate.

III. TRUST EVALUATION APPROACH

Trust evaluation in a cloud requires data of several parameters obtained from a real cloud computing environment. The unavailability of such practical environment's data makes any research problem more complex [22]. The problem is to evaluate the trust value of different service providers and design a model for the selection of CSPs while improving the QoS. We assume that all resources of a CSP stay in one data center. The proposed evaluation metrics are used to estimate the trust of a CSP.

A. TRUST EVALUATION MODEL

Trust is evaluated quantitatively in the proposed model. We use two steps to evaluate the trust of a CSP. One is workload modeling that involves the assessment of the arrival rates of requests for resources such as CPU, memory, bandwidth requirements placed by users. In the second step, system modeling aims at evaluating the performance of a cloud system at the allocation based on the performance of the service providers. The model estimates trust based on the value of the QoS metrics which are availability, reliability, data integrity, and time efficiency.

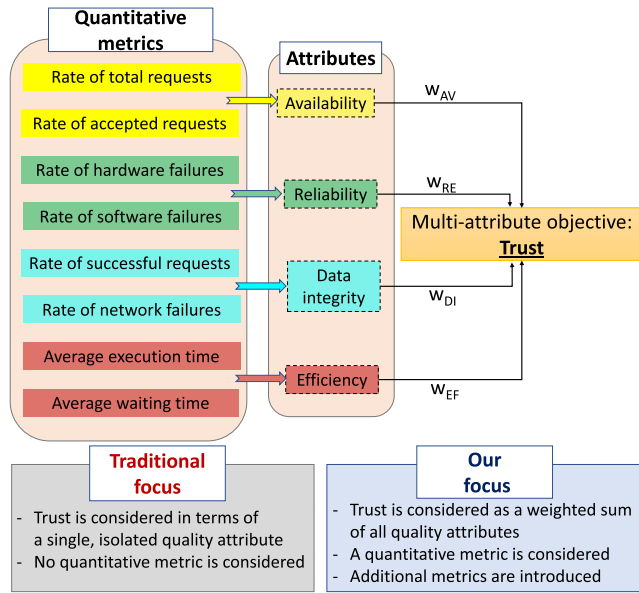


FIGURE 1. Proposed trust evaluation approach.

We compute trust based on the previous credentials of the CSPs using the four quality attributes and our proposed evaluation metrics shown in Fig. 1. Availability is one of the major concerning parameters in a cloud environment. Availability is calculated based on the resource availability of a CSP. All cloud service providers emphasize the availability of their service in their Service Level Agreement (SLA). Amazon, Google, Microsoft, IBM, and Salesforce maintain the availability of 99.9% which is still lower than the expectation [23]–[26]. The availability of a CSP is calculated based on the rate of the accepted and total requests. We consider a request accepted if the request queue can process the request within the prescribed time threshold. The availability of CSP n is calculated using the equation below where AC_{rq} is the rate of accepted requests and TO_{rq} is the rate of incoming total requests.

$$AV_n = 1 - \frac{AC_{rq}}{TO_{rq}} \quad (1)$$

Reliability ensures the ability of a service to keep functioning with a specific performance level for a specified time. Reliability of cloud is not addressed or appropriately considered by the previous cloud researchers [27], [28]. However,

due to many outages in a cloud, reliability issues have become a major concern in a cloud environment. We consider reliability as a failure-free operation. In terms of failures in the cloud, we consider hardware and software failures. Reliability is calculated based on the rate of successful and accepted requests. If a request completes the execution without any failure occurrence, we consider that request a successful one. The reliability of CSP n is calculated as below where SU_{rq} is the rate of successful requests and AC_{rq} is the rate of accepted requests.

$$RE_n = \frac{SU_{rq}}{AC_{rq}} \quad (2)$$

Data integrity considers data loss due to latency and network failures. If a service is unable to grow fast then elasticity failure occurs that will impact the quality of service. We consider the elasticity failure as a network failure as it occurs due to an error in measuring load. Data integrity is checked by estimating the data loss. We use the equation below to estimate the data integrity of CSP n where SU_{rq} is the rate of successful requests and NF_{rq} is the rate of requests where a data failure occurs and generates a data loss.

$$DI_n = \frac{SU_{rq} - NF_{rq}}{SU_{rq}} \quad (3)$$

In terms of efficiency, time efficiency is evaluated that considers the execution time and waiting time of each CSP. Efficiency is calculated based on the rate of promised and actual execution time. In an ideal case, the promised time (PR_t) should be equal to the execution time (EX_t) as the waiting time (WA_t) is supposed to be 0.

$$EF_n = \frac{PR_t}{WA_t + EX_t} \quad (4)$$

Finally, we combine all the individual rates multiplying with their corresponding weighting factors to compute the trust of a CSP as follows:

$$TR_n = W_{av}AV_n + W_{re}RE_n + W_{di}DI_n + W_{ef}EF_n \quad (5)$$

We repeat the same process for each of the CSP and evaluate the trust for all of them. The following architecture is used to evaluate the trust.

B. ARCHITECTURE USED IN SIMULATION

Cloud providers can have multiple data centers in different geographic locations [29]. The failure rates and reliability of servers have different values that exist on different racks. The primary cause of a rack failure is the interruption on the power distribution or the rack switches [30]. Each of the servers is a combination of resources such as CPU, storage, RAM, and bandwidth and each of these resources has different failure rates and reliability levels. An entire data center may fail due to a natural disaster or due to a prolonged power outage that can cause a network interruption, and as a consequence service outage can occur. In our model, for simulating a cloud architecture, we consider a layered software framework and

cloud components through CloudSim. The core functionalities are implemented at the lowest layer such as arriving and processing requests, resource creation (data centers, servers, VMs), communication among resources, and management of the load execution. We implement our customized VM allocation policy extending the functionality. We develop user requests, and resource availability, and different scenarios at this layer. We model a data center for handling users' requests. This request requires the allocation of VM to a data center's servers. In our considered architecture, the data center of a CSP is composed of many servers and is responsible for managing resources. Each server has a fixed amount of pre-configured resources for allocating to VMs [31]. The primary concerns in the cloud such as allocation of VMs, managing load execution, and monitoring are controlled at this layer. We used the aforementioned architecture for emulating a cloud environment and collected the necessary data to estimate the value of different attributes.

C. SIMULATOR

CloudSim simulator is used in the simulation due to its acceptance in the research community for representing real cloud environments [32]–[34]. To validate the proposed evaluation and integration models, it is essential to create a cloud using a simulator that can imitate practical cloud instances and failures by executing different loads in different scenarios. Failure injection and failure data collection in VM granularity levels are restricted to real cloud provider's configurations and infrastructures. To verify the effectiveness of the proposed models, modeling and simulations are used as an alternative. Our proposed approach creates an option to improve the performance of quality attributes in a cloud scenario. The problem modeling and simulation environments ensure the applicability and validity of the proposed approach in a real cloud environment. Moreover, CSPs can achieve benefits from the results by integrating new features and extensions to their infrastructures.

All the above-mentioned metrics are used in the trust evaluation model using the simulator. In the simulation, Poisson distribution is used for generating requests and occurring failures. Due to the popularity and applicability of Poisson distribution in modeling request and failure generation in cloud, it is the most appropriate distribution [35], [36], [37]. At first, the initialization of all required parameters for simulation is set to obtain the necessary data. Multiple environments with different problem sizes are being created in the simulator. We execute the model in distinct environments and workloads for validation. We run each of the experiments ten times and obtain the average value to evaluate all the required attributes.

IV. RESOURCE ALLOCATION MODEL

The objective of this model is to allocate resources in such a way that increases trust and decreases delay. Availability, reliability, data integrity, and efficiency are considered to obtain the trust of a CSP. Allocating resources in the cloud while maximizing the service quality is treated as one of the

complex problems. The resource allocation model integrates trust obtained from our trust evaluation model. The optimization model is designed for joint optimization criteria as a Mixed Integer Linear Programming (MILP) problem. The joint optimization aims at maximizing trust and minimizing delay to improve the service performance.

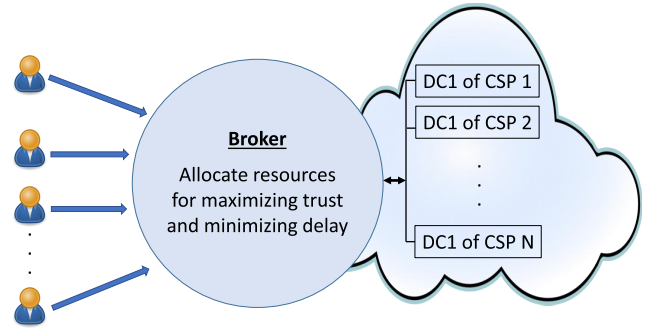


FIGURE 2. Maximizing trust in the resource allocation model.

An example of the proposed resource allocation process is shown in Fig. 2 with N CSPs. At first, users provide requests for resources to a broker. Broker has access to the value of trust and delay for all the data centers of all CSPs. The broker executes the proposed resource allocation model and allocates resources while maintaining the objectives and required constraints. The model aims for an optimized allocation of resources that maximizes trust to improve the quality of service.

A. PROBLEM FORMULATION

The research problem that we want to formulate is allocating resources in multiple CSPs while increasing trust and decreasing communication delay. The capacity of the servers located in the data center and total traffic are used in estimating communication delay. The description of all the notations of our model is presented in Table 1.

We allocate a set of servers for each of the CSP. A number of VMs can be allocated inside each of the servers based on the required amount of resources. A VM is created by applying the users' requirements for the combination of processing unit, storage, memory, and bandwidth. We use R to determine the amount of resources of a VM.

B. FORMULATION OF OBJECTIVE FUNCTIONS

We want to maximize f_{TR} , the overall trust value of the resource allocation. TR_{isn} is the trust of VM i in server s in CSP n , and A_{isn} is used as a flag to determine the allocation of VM i in server s in CSP n .

$$f_{TR} = \sum_{n \in \mathcal{N}} \sum_{s \in \mathcal{S}_n} \sum_{i \in \mathcal{V}} TR_{isn} A_{isn} \quad (6)$$

The communication delay of a server is evaluated using the values of the total traffic between VMs and the capacity of the server where the VMs will be placed. In DL_{isn} , we estimate the delay to allocate VM i of server s in CSP n . For measuring

TABLE 1. Description of the symbols.

Symbol	Description
V_i	VM i
T_{ij}	Communication Traffic of VMs
S_n	Number of servers in server s of CSP n
f_{TR}	Objective function for trust
f_{DL}	Objective function for communication delay
DL_{isn}	Delay for a specific VM i located in server s of CSP n
CP_{sn}	Server s 's capacity of CSP n
$\mathcal{R}$	The amount of resources
CA_{snr}	Capacity for a specific server s located in CSP n for estimating the amount of resource $r \in \mathcal{R}$
β_{isnr}	Evaluates the resource usage for VM i located in server s of CSP n
A_{isn}	The value of the flag variable equal to 1 represents the allocation of VM i in server s of CSP n

total traffic, we combine the communication of VM i with all other VMs of the corresponding server s . The delay of a VM i is evaluated by combining all the above traffic divided by the capacity of server s in CSP n using the equation below:

$$DL_{isn} = \frac{\sum_{i,j \in \mathcal{V}} T_{ij}}{CP_{sn}} \quad (7)$$

$$f_{DL} = \sum_{n \in \mathcal{N}} \sum_{s \in S_n} \sum_{i \in \mathcal{V}} DL_{isn} A_{isn} \quad (8)$$

$$\text{Minimize } \{-f_{TR}, f_{DL}\} \quad (9)$$

$$\text{Subject to : } \sum_{s \in S_n} \sum_{n \in \mathcal{N}} A_{isn} = 1 \quad \forall s \in S_n, \forall i \in \mathcal{V} \quad (10)$$

$$\sum_{i \in \mathcal{V}} \sum_{s \in S_n} \sum_{n \in \mathcal{N}} \beta_{isnr} A_{isn} \leq CA_{snr} \quad (11)$$

$$\forall r \in \mathcal{R}, \forall s \in S_n, \forall n \in \mathcal{N} \quad (12)$$

We set all the constraints in the above. With constraint 10, we confirm the allocation of one VM to one server of a CSP only. The limitations and availability of resources are maintained by constraint 11. Integrity constraint is fulfilled using constraint 12. In the literature, it is shown that such type of problems are NP-hard problems [38].

C. PROPOSED GENETIC ALGORITHM

We solve the joint optimization model using the optimizer varying several weight combinations to the objective functions. The variation of weights is required due to multiple reasons such as the requirements of the type of the application or QoS parameters. Due to the NP-hard nature of the problem, formal methods are suitable only for small-scale instances. To overcome this situation, a heuristic is proposed for generating an approximated near-optimal solution.

For solving the resource allocation problem, we propose a Genetic Algorithm that is usually applied in several machine learning approaches [39]. This type of meta-heuristic algorithm is appropriate to find a near-optimal solution in large-scale problem instances. The complexity of the three major operations of the genetic algorithm determines the total

Algorithm 1 Optimizing Trust and Delay in Resource Allocation

Input:

- Resource capacities for all servers of all CSPs
- Availability, reliability, data integrity, and time efficiency of the servers of all CSPs

- Matrix for communication traffic of all VMs

Output:

- The best VM allocation set with maximum fitness value CR_b

```

1: for all incoming requests  $rq \in \mathcal{RQ}$ 
2:   for all existing CSPs  $n \in \mathcal{N}$ 
3:     for all available servers  $s \in S_n$  do
4:       Estimate trust
5:       Estimate delay
6:     end for
7:   end for
8: end for
9:  $CR_b \leftarrow$  Initialize this set empty
10: Randomly generate a set of  $C$  chromosomes
11: Evaluate fitness (trust and delay) for the above  $C$  chromosomes
12: generation = 0
13: while generation <= MAXgeneration
14:   while new chromosomes set  $\neq C$ 
15:     Select two chromosomes applying roulette wheel operator
16:     Based on the crossover probability, crossover occurs
17:     According to the mutation probability, mutation occurs
18:   end while
19: Estimate fitness of the new set of chromosomes
20: Select the fittest chromosomes  $CR_n$  using selection operation
21:   if  $f(CR_n) > f(CR_b)$ 
22:      $CR_b = CR_n$ 
23:   end if
24: generation = generation + 1
25: end while
26: return the best allocation set ( $CR_b$ )

```

time complexity of such an algorithm. The time complexity for selection operation depends on the size of the request queue and it is $O(RQ)$. On the other hand, crossover and mutation operations have a time complexity of $O(N * V * RQ)$. We estimate the total complexity by combining the above operations and it is $O(N * V * RQ * MAX_{gen})$. The complexity of the algorithm provides the feasibility for implementing the model in a large-scale real cloud environment to ensure the allocation of VMs while jointly optimizing trust and delay.

We design the chromosome CR as a vector where the size of the vector depends on the requests. The size of the vector can be $S * N$ -dimensional while we keep the number of VMs the same in all requests. The value of the chromosome indicates the location of a VM to a server of a specific CSP.

We define trust and delay as the fitness of the chromosome. We estimate the trust and delay and combine them to compute the fitness. The fittest set of chromosomes represents the best solution for allocating VMs. We define the fitness function by combining the objective functions using a weighted sum technique. Multiple combinations of the weights are examined for the objective functions to determine the proper trade-off. We present the pseudo-code of the proposed algorithm in Algorithm 1. As input of the algorithm, it takes the following: available resource capacities, availability, reliability, data integrity, time efficiency, and traffic matrix of VMs for all servers and CSPs. At first, a random chromosome population of size C is generated and the algorithm calculates the fitness of each of them. After the fitness being evaluated, based on the fitness value selection, crossover, and mutation are applied and a new set of chromosomes are generated [40] (lines 12-20).

The roulette wheel method is used for selection operations. We use a two-point crossover process to have the better parts of parent chromosomes. In the process, the selected chromosomes exchange their parts and create two new child chromosomes. A single bit of the chromosome is changed in mutation operation, and thereby a new chromosome is generated. We assign the crossover and mutation rates and based on the values the operations occur in the algorithm. After obtaining a new generation of chromosomes, we estimate the fitness of each of them, and the set of chromosomes with maximum fitness value is selected as our new solution CR_n . If the fitness of this set is more than that of the previous set, the algorithm selects this set as the best solution CR_b . The algorithm will return the best set of solutions when it reaches the termination conditions. In our algorithm, we set the maximum number of generations $MAX_{generation}$ to terminate the proposed algorithm.

The proposed optimization problem is modeled with several constraints and the constraints handling method depends on the application of selection operation. We enforce several rules while comparing two solutions such as for any solution in terms of feasibility, we discard the infeasible solution and choose the feasible one. The algorithm finally returns CR_b , the best VM allocation set that maximizes trust and minimizes delay.

V. SIMULATIONS AND RESULTS

The details of the experiments and results analysis are discussed in this section. CloudSim [31] is used for cloud data collection, and MATLAB is used for the optimized resource allocation in the experiments. We perform the experiments in multiple scenarios for testing the applicability of the approach. We implement the trust evaluation model using CloudSim and obtain the required data to calculate the trust of a CSP. Finally, the performance of the optimization model for resource allocation is compared with the exact optimized solution and the performance of the proposed heuristic is presented.

We obtain the necessary cloud data to compute trust from the experiments executed in the CloudSim environment. Based on the collected data, we calculate the trust of different CSPs. We use the trust value of our model and integrate this in the optimization model to execute the optimizer and GA in MATLAB environments to analyze the performance. To run experiments, we use an Intel Core i7 machine with a 4 GHz processor. In GA, we set the population size based on our requirements and generation size to 1000.

We execute the proposed model in four distinct scenarios. The number of servers and CSPs are varied in each of the scenarios. We modify the number of servers and CSPs from 30 to 60 and 6 to 12 respectively. In each scenario, the number of servers is increased by 10 and CSPs by 2. The above scenarios may represent real scenarios as the number of CSPs that can participate together cannot be more as several constraints need to be fulfilled.

The trust of a CSP is estimated using the weighted sum of the value of availability, reliability, data integrity, and time efficiency. Fig. 3 depicts the trend of the availability metric with respect to different loads. The number of incoming requests and executing loads are varied, and three distinct cases are considered. A small load represents the number of requests and executing loads are minimum, on the other hand, there is one case with a maximum or heavy load. In between the minimum and maximum loads, we also consider one case which we name medium load. The trend in the figure shows that with a small load the availability is maximum and with a heavy load it is minimum. For calculating the availability value, we consider several such scenarios and take the average of 10 executions of those scenarios for the availability value.

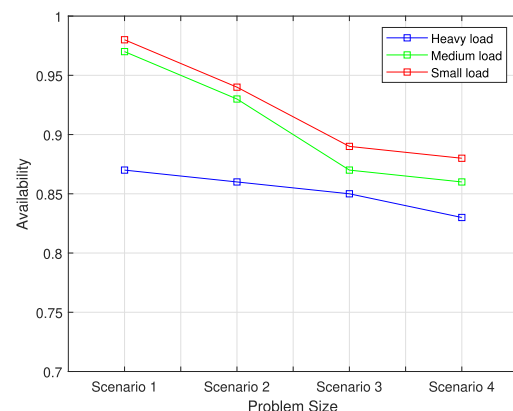


FIGURE 3. Resources availability with different loads.

For calculating reliability, we consider some of the hardware and software failures and combine them together to get the reliability attribute. In case of hardware failures, failures of virtual machines and processing elements are considered. The required software that is involved to process requests and execute workload is considered software failures. We execute several experiments with variations in the requests and loads

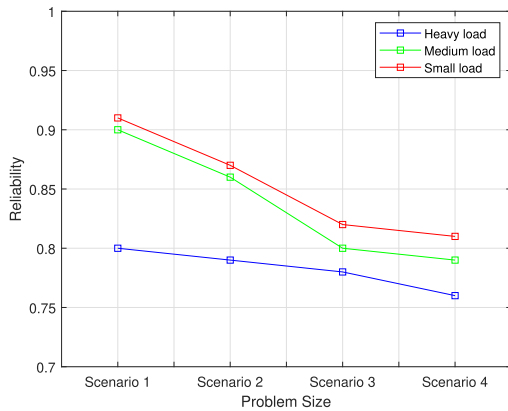


FIGURE 4. Reliability of resources with different loads.

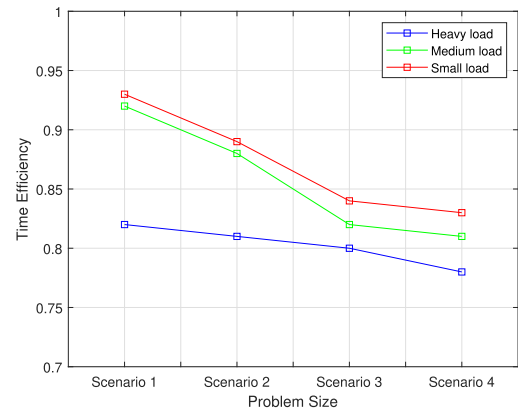


FIGURE 6. Time efficiency with different loads.

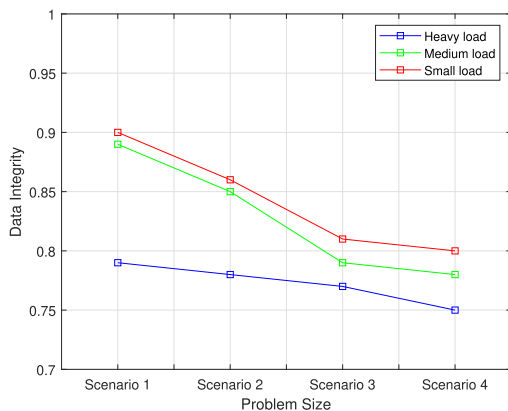


FIGURE 5. Data integrity with different loads.

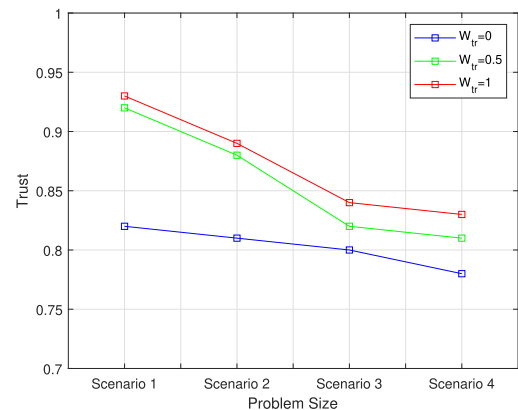


FIGURE 7. Trust with different weight values.

and show that in Fig. 4. The reliability trend depicts that failures are more frequent in the case of a maximum number of requests and loads.

Fig. 5 shows the trend of the value of data integrity that was varied with the variations of the different loads. Fig. 6 confirms the efficiency in terms of the required time for all the different scenarios. In an ideal case, it is supposed to be 1 but practically we can try to get closer to the maximum possible value. After obtaining all the four attributes, we take the averages of all the attributes and combined them together as a weighted sum to get the trust value of one CSP.

Fig. 7 shows an important performance trend of the approach on the basis of trust value. The trust value is calculated using the weighted sum of availability, reliability, data integrity, and time efficiency. The results show the variation in the values of trust of our model with a variation in the weight. A 0.25 value in the weight of trust means trust is considered partially but the delay has more weight which is 0.75 in the objective function. Similarly, the other side is also considered in the objective function. A weight value of 0.5 indicates an equal weight on both of the objective functions. Fig. 7 shows that the trust of the model improves approximately 10% in all distinct scenarios. This performance gain ensures the validity of the approach.

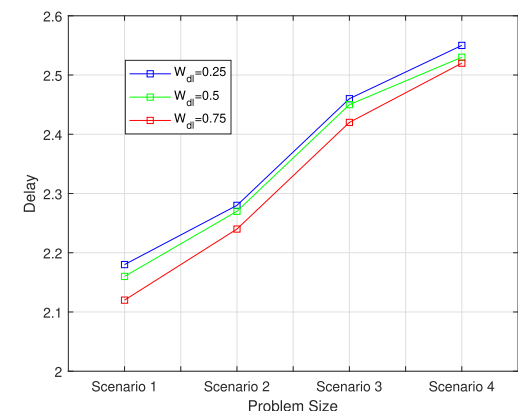


FIGURE 8. Delay with different weight values.

Fig. 8 depicts the trends of delay. The same weight variations are considered to show the variations in the values of communication delay. Applying the model delay is decreased by approximately 10%. The performance improvement is not massive but even a small reduction of delay improves the quality of service. The results can be more prominent in the real environment.

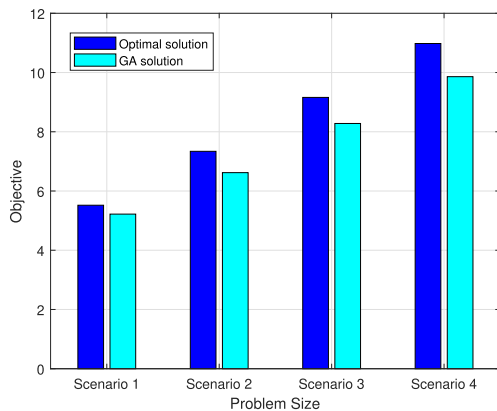


FIGURE 9. Objective value comparison between optimal and genetic algorithm.

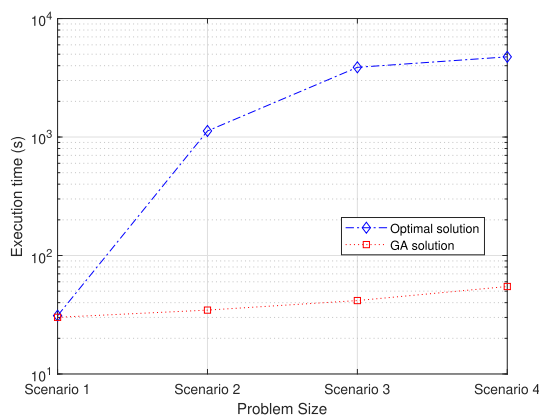


FIGURE 10. Execution time comparison between optimal and genetic algorithm.

Fig. 9 shows one of the principal performance trends of the approach that is the objective value. The objectives of trust and delay are combined to calculate the objective value. The obtained value of the objective in the approach is nearly the same as an optimal solution. The aim is to maximize trust and minimize delay, and the proposed GA achieves almost the same objective values as an optimizer. The performance trend remains the same while varying the size of the problem in terms of CSPs and servers. This trend confirms that the proposed model is appropriate in a large cloud environment.

Fig. 10 also shows the performance of the approach on the basis of execution time. Results depict a linear increase in the execution time with an increase in the number of CSPs and servers. The above trend confirms the usability of the approach in real scenarios. The results exhibit that increase in the problem size does not require any noticeable execution time. This trend provides the assurance of the approach while increasing trust and decreasing delay. Moreover, the execution time of the approach is more significant compare to the optimizer in a large scenario.

Fig. 11 reveals the Pareto front obtained from the proposed heuristic. The trade-off between the objective functions is depicted using the Pareto front. Allocating resources in the

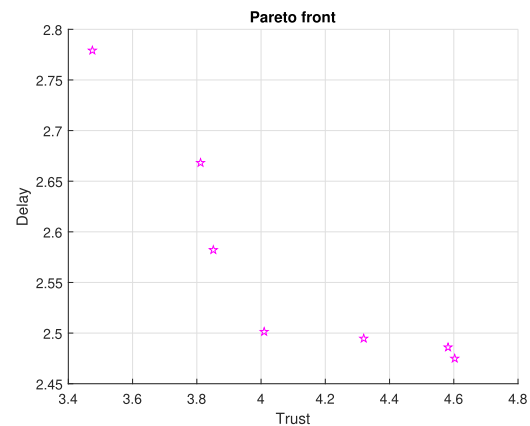


FIGURE 11. Pareto font between trust and delay.

most trusted CSPs may increase trust but it will acquire a massive delay. Distributing resources will reduce delay as the load will be divided but trust will be decreased.

The performance results show that the trust of different CSPs is effectively estimated while varying the simulation environments. The performance results exhibit that the proposed approach can be an appropriate solution for optimizing resource allocation in a large cloud environment while fulfilling the objectives. The improvement of the performance can be more prominent with an application in a real large cloud environment.

VI. CONCLUSION

The contributions of this work can be outlined into three main parts: evaluation of trust of cloud computing, integration of trust in resource allocation model for improving QoS, and the application of Genetic algorithm to allocate resources effectively in a reasonable amount of time. The first contribution is the trust evaluation in multi-cloud settings. These include consideration of proper attributes and metrics and the quantification of the metrics for estimating trust. The evaluation is shown in several environments for providing practical solutions. The trust evaluation is crucial in the adoption of a multi-cloud environment. The proposed evaluation model will attract more users as it estimates the trust of the cloud more accurately. This type of evaluation model will be of high importance to both service providers and customers.

The performance analysis of the resource allocation model depicts that the model effectively allocates resources while maintaining the objectives. Applying a Genetic algorithm to get an optimal solution in a reasonable amount of time is our third contribution. The execution time of the algorithm to find a solution requires significantly less time compared to an optimizer and, this feature validates the applicability of the method. Achieving the same performance trend while varying numbers of resources confirms that the improvement of the performance will not be reduced even if a real cloud environment is used with a large number of resources. In the future, we plan to provide a reliable and secure model for trust management among different cloud service providers.

REFERENCES

- [1] E. Buer and R. Adams, *Reliability and Availability of Cloud Computing*. Hoboken, NJ, USA: Wiley, 2012.
- [2] A. B. M. B. Alam, T. Halabi, A. Haque, and M. Zulkernine, "Multi-objective interdependent VM placement model based on cloud reliability evaluation," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Dublin, Ireland, Jun. 2020, pp. 1–7.
- [3] K. V. Vishwanath and N. Nagappan, "Characterizing cloud computing hardware reliability," in *Proc. 1st ACM Symp. Cloud Comput. (SoCC)*, 2010, pp. 193–204.
- [4] H. Liao, Z. Zhou, X. Zhao, L. Zhang, S. Mumtaz, A. Jolfaei, S. H. Ahmed, and A. K. Bashir, "Learning-based context-aware resource allocation for edge-computing-empowered industrial IoT," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4260–4277, May 2020.
- [5] A. Musaddiq, Y. B. Zikria, O. Hahm, H. Yu, A. K. Bashir, and S. W. Kim, "A survey on resource management in IoT operating systems," *IEEE Access*, vol. 6, pp. 8459–8482, 2018.
- [6] P. Pradhan, P. K. Behera, and B. N. B. Ray, "Improved max-min algorithm for resource allocation in cloud computing," in *Proc. 6th Int. Conf. Parallel, Distrib. Grid Comput. (PDGC)*, Nov. 2020, pp. 22–24.
- [7] Z. Xuejie, W. Zhijian, and X. Feng, "Reliability evaluation of cloud computing systems using hybrid methods," *Intell. Autom. Soft Comput.*, vol. 19, no. 2, pp. 165–174, Apr. 2013.
- [8] N. Padmapriya and R. Rajmohan, "Reliability evaluation suite for cloud services," in *Proc. 3rd Int. Conf. Comput., Commun. Netw. Technol. (ICCCNT)*, Jul. 2012, pp. 1–6.
- [9] B. Yang, F. Tan, Y.-S. Dai, and S. Guo, "Performance evaluation of cloud service considering fault recovery," in *Proc. IEEE Int. Conf. Cloud Comput.*, Dec. 2009, pp. 571–576.
- [10] H. R. Faragardi, R. Shojaei, H. Tabani, and A. Rajabi, "An analytical model to evaluate reliability of cloud computing systems in the presence of QoS requirements," in *Proc. IEEE/ACIS 12th Int. Conf. Comput. Inf. Sci. (ICIS)*, Niigata, Japan, Jun. 2013, pp. 315–321.
- [11] Y. Hu, J. Wong, G. Iszlai, and M. Litoiu, "Resource provisioning for cloud computing," in *Proc. Conf. Center Adv. Stud. Collaborative Res.*, 2009, pp. 101–111.
- [12] J. Praveenchandar and A. Tamarasi, "Dynamic resource allocation with optimized task scheduling and improved power management in cloud computing," *J. Ambient Intell. Humanized Comput.*, vol. 12, no. 3, pp. 4147–4159, Mar. 2021.
- [13] A. K. Bashir, R. Arul, S. Basheer, G. Raja, R. Jayaraman, and N. M. F. Qureshi, "An optimal multitier resource allocation of cloud RAN in 5G using machine learning," *Trans. Emerg. Telecommun. Technol.*, vol. 30, no. 8, p. e3627, Aug. 2019.
- [14] Z. Xiao, W. Song, and Q. Chen, "Dynamic resource allocation using virtual machines for cloud computing environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 24, no. 6, pp. 1107–1117, Jun. 2013.
- [15] W. Lin, J. Z. Wang, C. Liang, and D. Qi, "A threshold-based dynamic resource allocation scheme for cloud computing," *Procedia Eng.*, vol. 23, pp. 695–703, Jan. 2011.
- [16] R. K. Grewal and P. K. Pateriya, "A rule-based approach for effective resource provisioning in hybrid cloud environment," in *New Paradigms in Internet Computing*. Berlin, Germany: Springer, 2013, pp. 41–57.
- [17] J. Luo, W. Song, and L. Yin, "Reliable virtual machine placement based on multi-objective optimization with traffic-aware algorithm in industrial cloud," *IEEE Access*, vol. 6, pp. 23043–23052, 2018.
- [18] X. Wang, H. Gu, and Y. Yue, "The optimization of virtual resource allocation in cloud computing based on RBPSO," *Concurrency Comput., Pract. Exper.*, vol. 32, no. 16, p. e5113, Aug. 2020.
- [19] T. Halabi, M. Bellaiche, and A. Abusitta, "Online allocation of cloud resources based on security satisfaction," in *Proc. 17th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun./12th IEEE Int. Conf. Big Data Sci. Eng. (TrustCom/BigDataSE)*, Aug. 2018, pp. 379–384.
- [20] F.-H. Tseng, X. Wang, L.-D. Chou, H.-C. Chao, and V. C. M. Leung, "Dynamic resource prediction and allocation for cloud data center using the multiobjective genetic algorithm," *IEEE Syst. J.*, vol. 12, no. 2, pp. 1688–1699, Jun. 2018.
- [21] P. Gupta and S. P. Ghrera, "Power and fault aware reliable resource allocation for cloud infrastructure," *Procedia Comput. Sci.*, vol. 78, pp. 457–463, Jan. 2016.
- [22] A. B. M. B. Alam, T. Halabi, A. Haque, and M. Zulkernine, "Optimizing virtual machine migration in multi-clouds," in *Proc. Int. Symp. Netw., Comput. Commun. (ISNCC)*, Montreal, QC, Canada, Oct. 2020, pp. 1–7.
- [23] (2013). *Windows Azure*. Accessed: Apr. 2019. [Online]. Available: <http://www.windowsazure.com/en-us/support/legal/sla/>
- [24] (2020). *Amazon EC2*. Accessed: May 2020. [Online]. Available: <http://aws.amazon.com/ec2-sla/>
- [25] (2020). *Google Gmail*. Accessed: May 2020. [Online]. Available: <https://developers.google.com/storage/docs/sla>
- [26] (2020). *Salesforce*. Accessed: May 2020. [Online]. Available: <https://www.salesforce.com/editions-pricing/service-cloud/>
- [27] O. Beaumont, L. Eyraud-Dubois, and H. Larchevêque, "Reliable service allocation in clouds," in *Proc. IEEE 27th Int. Symp. Parallel Distrib. Process.*, May 2013, pp. 55–66.
- [28] A. B. M. B. Alam, M. Zulkernine, and A. Haque, "A reliability-based resource allocation approach for cloud computing," in *Proc. IEEE 7th Int. Symp. Cloud Service Comput. (SC)*, Kanazawa, Japan, Nov. 2017, pp. 249–252.
- [29] E. Bauer and R. Adams, *Reliability and Availability of Cloud Computing*. Hoboken, NJ, USA: Wiley, 2012.
- [30] M. Jammal, A. Kanso, and A. Shami, "CHASE: Component high availability-aware scheduler in cloud computing environment," in *Proc. IEEE 8th Int. Conf. Cloud Comput.*, Jun. 2015, pp. 477–484.
- [31] R. Buyya, R. Ranjan, and R. N. Calheiros, "Modeling and simulation of scalable cloud computing environments and the CloudSim toolkit: Challenges and opportunities," in *Proc. Int. Conf. High Perform. Comput. Simulation*, Jun. 2009, pp. 1–11.
- [32] F. M. Alturkistani and S. S. Alaboodi, "An analytical model for availability evaluation of cloud service provisioning system," *Int. J. Adv. Comput. Sci. Appl.*, vol. 8, no. 6, pp. 240–247, 2017.
- [33] B. G. Batista, J. C. Estrella, C. H. G. Ferreira, D. M. L. Filho, L. H. V. Nakamura, S. Reiff-Marganiec, M. J. Santana, and R. H. C. Santana, "Performance evaluation of resource management in cloud computing environments," *PLoS ONE*, vol. 10, no. 11, 2015, Art. no. e0141914.
- [34] S. Meng, X. Qiu, L. Luo, H. Xu, and M. Lei, "Reliability simulation in cloud computing system," *Int. J. Performability Eng.*, vol. 14, no. 9, p. 2015, 2018.
- [35] A. B. M. B. Alam, T. Halabi, A. Haque, and M. Zulkernine, "Reliability-based formation of cloud federations using game theory," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Taipei, Taiwan, Dec. 2020, pp. 1–6.
- [36] Y.-S. Dai, B. Yang, J. Dongarra, and G. Zhang, "Cloud service reliability: Modeling and analysis," in *Proc. 15th IEEE Pacific Rim Int. Symp. Dependable Comput.*, Nov. 2009, pp. 1–17.
- [37] N. Limrungrui, J. Zhao, Y. Xiang, T. Lan, H. H. Huang, and S. Subramaniam, "Providing reliability as an elastic service in cloud computing," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2012, pp. 2912–2917.
- [38] H. Kellerer, U. Pferschy, and D. Pisinger, "Multidimensional knapsack problems," in *Knapsack Problems*. Berlin, Germany: Springer, 2004, pp. 235–283.
- [39] T. Murata and H. Ishibuchi, "MOGA: Multi-objective genetic algorithms," in *Proc. IEEE Int. Conf. Evol. Comput.*, vol. 1, Nov. 1995, pp. 289–294.
- [40] A. E. Eiben and J. E. Smith, *Introduction to Evolutionary Computing* (Natural Computing Series). New York, NY, USA: Springer-Verlag, 2008.



A. B. M. BODRUL ALAM (Member, IEEE) received the B.Sc. degree (Hons.) in computer science from Jahangirnagar University, Bangladesh, in 2005, the M.Sc. degree from the Department of Computer Science, Ryerson University, Canada, in 2012, and the Ph.D. degree from Queen's University, Canada, in 2020. He is currently working as a Postdoctoral Researcher with the Department of Computer Science, Lakehead University, Thunder Bay, ON, Canada. Before joining

Ryerson University, he served as a full-time Lecturer with the Department of Computer Science, Stamford University, Bangladesh, for five years. He has published several peer-reviewed journals and conference papers. He also collaborated with two other universities: Western University and the University of Winnipeg while conducting his research. His research interests include optimization, mathematical modeling, reliability analysis, reliable resource management, VM placement, VM migration, survivable network design, and cloud computing.



ZUBAIR MD. FADLULLAH (Senior Member, IEEE) was an Associate Professor at the Graduate School of Information Sciences (GSIS), Tohoku University, Japan, from 2017 to 2019. He is currently an Associate Professor with the Computer Science Department, Lakehead University, and the Research Chair of Thunder Bay Regional Health Research Institute (TBRHRI), Thunder Bay, ON, Canada. His research interests include the areas of emerging communication systems, such as 5G new radio and beyond, deep learning applications on solving computer science and communication system problems, UAV-based systems, smart health technology, cyber security, game theory, smart grid, and emerging communication systems. He is a Senior Member of IEEE Communications Society (ComSoc). He received several best paper awards at conferences, including IEEE/ACM IWCMC, IEEE GLOBECOM, and IEEE IC-NIDC. He is currently an Editor of IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY (TVT), *IEEE Network* magazine, IEEE ACCESS, IEEE OPEN JOURNAL OF THE COMMUNICATIONS SOCIETY, and *Ad Hoc & Sensor Wireless Networks* (AHSWN) journal.



SALIMUR CHOUDHURY (Senior Member, IEEE) received the Ph.D. degree in computing from Queen's University, Kingston, ON, Canada, in 2012. He is currently an Assistant Professor with the Department of Computer Science, Lakehead University, Thunder Bay, ON. He is also the Director of the Lakehead Optimization Research Group (LORG). His research interests include designing algorithms for wireless communications, optimization, cellular automata, and approximation algorithms. He was the Technical Program Chair of the SGIoT 2017, SGIoT 2018, and iThings 2018 conferences. He is also an Editor of *Parallel Processing Letters*.

...